

Configuration and Parameter Reference Manual

JSON Configuration Parameters for the Optimization Framework

Introduction

This document lists JSON-configurable parameters that are read directly, propagated into stage dictionaries, or applied indirectly through the global `pill.glob` state used by the optimization and feature-definition layers. The scope is intentionally limited to configuration parameters that can affect optimization behavior, geometry, numerical stability, convergence, constraints, heuristics, or refinement.

Interpretation rules used in this reference.

- **Global-to-stage inheritance:** before each optimization stage, the code overlays stage-local keys such as `transition`, `extension`, `boundary`, `combine`, `p`, `n`, `bounds`, and `beta` onto the global state. Therefore, a stage entry overrides the corresponding global setting only for that stage.
- **Approximate locations:** line numbers are approximate and refer to the supplied source files, especially functions `load_config()`, `run_optimization_stage()`, `build_constraints_from_config()`, `greedy_additive_refinement()`, and `run_configured_optimization()` in `optimize.py`.

1. Top-level configuration parameters

Table 1: Top-level JSON parameters — technical reference

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
start_strategy	start_strategy	optimize.py, run_configured_optimization() around lines 2186–2190; used by initialize_features()	string; at least cross; other values depend on implementation	cross
num_features	num_features	optimize.py, run_configured_optimization() around lines 2186–2190	integer ≥ 0	8
start_radius	start_radius	optimize.py, run_configured_optimization() around line 2191; consumed by initialize_features()	float > 0	0.1
random_seed	random_seed	optimize.py, run_configured_optimization() around line 2192; consumed by initialize_features()	integer or null	11
initial_s0	initial_s0	optimize.py, run_configured_optimization() around line 2093	array of floats	None
testcase	testcase	optimize.py, run_configured_optimization() around lines 2083–2157	string	None / absent
log_level	log_level (top-level)	optimize.py, load_config() lines 631–647; logger at lines 620–628	string enum	info
global	global	optimize.py, run_configured_optimization() around lines 1345, 2071, 2177, 2210 ff.; applied through pill.glob and stage inheritance	object	empty object if absent
constraints	constraints	optimize.py, build_constraints_from_config() around lines 1232–1237; passed into stage configs around line 2212	object	empty object if absent
orientation_run	orientation_run	optimize.py, run_configured_optimization() around lines 2205 ff.	array of objects	empty list if absent

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
convergence_run	convergence_run	optimize.py, run_configured_optimization() around lines 2223 ff.	array of objects	empty list if absent
heuristics	heuristics	optimize.py, used after main stages in run_configured_optimization() and in apply_heuristics()	object	empty object if absent
additive	additive	optimize.py, greedy_additive_refinement() around line 1334 and subsequent pipeline calls	object	empty object if absent

Table 2: Top-level JSON parameters — semantic reference

Parameter Name	Description / meaning	Typical effect
start_strategy	Initialization strategy for the initial pill placement when no explicit <code>initial_s0</code> is supplied. Example shows <code>cross</code> . The exact accepted strategy strings depend on branches inside <code>initialize_features()</code> .	Controls initial feature geometry and therefore the basin of attraction of the nonlinear optimization. Strongly affects convergence speed and final reconstruction quality.
num_features	Number of pill features in the initial design vector. Each feature contributes five optimization variables (P_x, P_y, Q_x, Q_y, r)	Larger values increase representational capacity but also problem dimension, computational cost, and risk of over-parameterization.
start_radius	Initial radius assigned to newly created pill features.	Larger values produce broader initial support and smoother initial overlap; smaller values start more locally but can slow early coverage.
random_seed	Seed for reproducible randomized initialization. Only relevant if the chosen start strategy contains random placement.	Fixes initialization reproducibility, which is valuable for comparing solver settings and staged schedules.
initial_s0	Optional explicit initial design vector. If present, it bypasses automatic feature initialization. Must be consistent with the expected <code>5*num_features</code> structure.	Allows warm-starting from a prior run or externally generated design. Particularly useful for continuation studies or debugging.

Parameter Name	Description / meaning	Typical effect
<code>testcase</code>	Internal/testing mode selector used to replace the normal density-loading path with synthetic or special-case inputs. Visible branch names include <code>random_feature</code> , <code>0</code> , <code>P</code> , <code>N</code> , <code>extreme</code> , <code>extreme_boxextension</code> , <code>random_feature2</code> , <code>smaller_box</code> , <code>featurecrossing</code> , <code>featurecrossing2</code> , and <code>from_image</code> .	Alters the target field and sometimes the initialization path. Intended mainly for experiments and regression tests rather than standard production use.
<code>log_level</code>	Global verbosity level applied during configuration loading. Visible levels are <code>none</code> , <code>warning</code> , <code>info</code> , <code>debug</code> .	Controls runtime console output and thus diagnostic visibility during staged optimization.
<code>global</code>	Container for baseline numerical, geometric, aggregation, and logging settings. These values are inherited by later stages unless a stage-local override exists.	Establishes the default global discretization and aggregation environment used by density evaluation and derivatives.
<code>constraints</code>	Container for geometric and other optimization constraints. Currently the visible active implementation is the segment-length family.	Enables geometric regularization and may prevent degenerate or unrealistic capsule geometries.
<code>orientation_run</code>	Ordered list of optimization stages for early “orientation” optimization. These stages are typically run with <code>reward_only=true</code> and optionally fixed radii.	Provides coarse geometric alignment before the final convergence stages. Useful for stabilizing difficult initializations.
<code>convergence_run</code>	Ordered list of later optimization stages aimed at final error reduction. Usually run after orientation stages with full objective evaluation.	Governs the fine-tuning schedule, final tolerance, and stagewise continuation in transition width, extension, or aggregation sharpness.
<code>heuristics</code>	Container for pruning and merging heuristics executed after the main optimization stages.	Can reduce redundant features, simplify geometry, and sometimes improve robustness of later refinement.
<code>additive</code>	Container for optional greedy additive refinement, including seeding thresholds and three nested stage definitions.	Enables automatic insertion of new features in poorly reconstructed regions, potentially improving accuracy at increased runtime.

2. Global settings (global)

Table 3: Technical reference for parameters in the `global` block

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
boundary	global - boundary	optimize.py, stage propagation around lines 868–870; feature_definition.py, Global.boundary property and boundary mapping in Pill.compute_boundary_value()	string; examples include smoothstep; exact set depends on feature_definition.py	smoothstep_extended
transition	global - transition	optimize.py, stage propagation around lines 868–870; feature_definition.py, Global.transition	float > 0	0.1
n	global - n	optimize.py, target sizing around line 2071 and stage override around lines 871–875; feature_definition.py, Global.n	array of two integers	[100,100]
p	global - p	optimize.py, stage propagation around lines 868–870; feature_definition.py, combine_p_norm()	positive float or integer	9
combine	global - combine	optimize.py, stage propagation around lines 868–870; feature_definition.py, combine_designs()	string enum	"p-norm"
order	global - order	optimize.py, line 2075; feature_definition.py, Global.order and integration-point combination	integer ≥ 1	3
k	global - k	optimize.py, stage propagation around lines 868–870; feature_definition.py, Global.k	float	7
N	global - N	optimize.py, stage propagation around lines 868–870; feature_definition.py, Global.N, smoothstep and cosine-weighted utilities	integer	3
beta	global - beta	optimize.py, stage propagation around lines 868–870; feature_definition.py, softmax / soft-cap style aggregators	positive float	18

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
extension	global - extension	optimize.py, stage propagation around lines 868–870; feature_definition.py, Global.extension	float	1.4
sum_cap	global - sum_cap	feature_definition.py, soft-capped sum aggregator (combine_sum_softcap()); applied via global state	float	1.1
bounds	global - bounds	optimize.py, around lines 2177 and 876–877; feature_definition.py, Global.bounds	object with x, y arrays	"x": [0,1], "y": [0,1]
x	global - bounds.x	optimize.py, line 2177; feature_definition.py, Global.bounds	two-element float array	[0,1]
y	global - bounds.y	optimize.py, line 2177; feature_definition.py, Global.bounds	two-element float array	[0,1]
log_level	global - log_level	example_run.json; interaction with optimize.py is schema-dependent	string enum	"info"

Table 4: Semantic meaning and optimization impact of parameters in the `global` block

Parameter Name	Description / meaning	Typical effect
boundary	Selects the boundary-to-density transition law used for each pill. Example uses <code>smoothstep</code> . The feature module visibly supports smoothstep-based, sigmoid-like, shifted smoothstep, and beta-shaped variants.	Changes the smoothness and sharpness of the capsule boundary, which directly affects gradients, Hessians, and optimization conditioning.
transition	Half-width of the smoothing band around pill boundaries. In effect, it defines how diffuse the geometric transition region is.	Larger values smooth the objective and derivatives, often improving robustness but blurring geometry; smaller values sharpen the reconstruction but can make optimization stiffer.
n	Grid resolution $[n_x, n_y]$ used for density rasterization and derivative evaluation.	Higher resolution improves geometric fidelity and derivative accuracy but increases objective cost, memory use, and Hessian assembly expense.
p	Exponent for <code>p-norm</code> aggregation of individual feature densities. Only relevant when <code>combine="p-norm"</code> .	Larger <code>p</code> makes the aggregation closer to a max-operator, which sharpens feature competition but may worsen conditioning. Smaller values yield smoother blending.

Parameter Name	Description / meaning	Typical effect
<code>combine</code>	Selects the feature-density aggregation rule. Visible implementations include p-norm , softmax/log-sum-exp, clamped sum, harmonic mean, cosine-weighted cap, and a soft-capped sum variant.	One of the most influential modeling choices: it changes how overlapping features interact and therefore alters objective smoothness, saturation behavior, and derivative structure.
<code>order</code>	Quadrature or sampling order used when averaging aggregator outputs over subcell integration points.	Higher order can reduce quadrature error and improve accuracy of objective/derivative evaluation, but increases per-evaluation cost.
<code>k</code>	Shape/sharpness parameter used by sigmoid-type boundary mappings. Only relevant for boundary laws that consume it.	Larger values generally sharpen the transition and may increase nonlinearity; smaller values make the transition milder.
<code>N</code>	Order parameter for smoothstep-like transition functions or related capping laws.	Alters smoothness class and steepness of the transition function; useful for balancing smooth derivatives against geometric sharpness.
<code>beta</code>	Temperature/sharpness parameter used by softmax or soft-capped sum style aggregators.	Higher beta makes soft approximations behave more like a hard maximum or hard cap; lower beta smooths the landscape.
<code>extension</code>	Additional extension of the transition band or effective evaluation box around each feature, depending on the active boundary law.	Larger values allow broader influence and smoother continuation stages; smaller values tighten the reconstruction to the pill geometry.
<code>sum_cap</code>	Cap level T for the soft-capped sum aggregator. Only meaningful when the selected aggregation rule uses a total-density cap.	Limits excessive density accumulation from overlapping features and can stabilize optimization when many features overlap.
<code>bounds</code>	Bounding box of the design domain. Stored as separate x and y intervals.	Changes the physical embedding of features and grid spacing; essential when optimizing on domains other than the unit square.
<code>x</code>	Lower and upper domain bounds in x	Rescales/relocates the horizontal domain and therefore the interpretation of feature endpoints.
<code>y</code>	-direction. Lower and upper domain bounds in y -direction.	Rescales/relocates the vertical domain and therefore the interpretation of feature endpoints.

Parameter Name	Description / meaning	Typical effect
<code>log_level</code>	Verbosity setting placed under <code>global</code> in the example file. The supplied code visibly reads top-level <code>log_level</code> ; therefore this field should be treated as <i>implementation-dependent</i> unless the surrounding code copies all global keys into <code>pill.glob</code> .	Intended to control logging verbosity. Verify whether the active code path expects top-level or nested placement.

3. Constraint settings (constraints)

Table 5: Technical parameters in the `constraints` block

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
<code>segment_length</code>	<code>constraints - segment_length</code>	<code>optimize.py</code> , <code>build_constraints_from_config()</code> around lines 1232–1237; <code>optimization_definition.py</code> , <code>segment-length</code> constraint classes	object	empty object if absent
<code>enabled</code>	<code>constraints - segment_length.enabled</code>	<code>optimize.py</code> , <code>build_constraints_from_config()</code>	boolean	<code>true</code>
<code>l_min</code>	<code>constraints - segment_length.l_min</code>	<code>optimization_definition.py</code> , <code>SegmentLengthConstraint</code> ; instantiated from <code>build_constraints_from_config()</code>	float ≥ 0	0.05
<code>l_max</code>	<code>constraints - segment_length.l_max</code>	<code>optimization_definition.py</code> , <code>SegmentMaxLengthConstraint</code> ; wired from <code>build_constraints_from_config()</code>	float > 0	2
<code>enforce_both</code>	<code>constraints - segment_length.enforce_both</code>	<code>optimize.py</code> , line around 1236	boolean	<code>false</code>
<code>angle</code>	<code>constraints - angle</code>	Appears in <code>example_run.json</code> ; no active implementation visible in supplied code	object	<code>inactive</code>
<code>enabled</code>	<code>constraints - angle.enabled</code>	Example only; no active reader found	boolean	<code>false</code>
<code>theta_star_deg</code>	<code>constraints - angle.theta_star_deg</code>	Example only; no active reader found	float	30
<code>delta</code>	<code>constraints - angle.delta</code>	Example only; no active reader found	float	0.02
<code>overlap</code>	<code>constraints - overlap</code>	Appears in <code>example_run.json</code> ; no active implementation visible in supplied code	object	<code>inactive</code>
<code>enabled</code>	<code>constraints - overlap.enabled</code>	Example only; no active reader found	boolean	<code>false</code>

Table 6: Descriptions and effects of parameters in the `constraints` block

Parameter Name	Description / meaning	Typical effect
<code>segment_length</code>	Container for lower/upper bounds on pill segment length $\ P - Q\ $	Prevents degenerate or excessively long capsules and may regularize the geometry during orientation and convergence stages.
<code>enabled</code>	. This is the clearly implemented geometric constraint family in the supplied code. Enables construction of segment-length constraints.	Turns geometric length constraints on or off.
<code>l_min</code>	Minimum admissible segment length. Enforces $\ P - Q\ \geq l_{\min}$	Avoids collapsing a pill into an almost circular point-like feature; supports numerical stability and interpretability.
<code>l_max</code>	. Maximum admissible segment length. Enforces $\ P - Q\ \leq l_{\max}$	Limits over-elongated features, which can otherwise dominate or cover unrealistic spans during orientation.
<code>enforce_both</code>	. Stage-handling flag for the segment-length family. Based on visible code, it influences whether both min and max constraints are enforced in stages that otherwise behave differently under <code>reward_only</code> .	Can make early stages more restrictive; useful if one wants to preserve both lower and upper geometric admissibility throughout the full pipeline.
<code>angle</code>	Placeholder block for an angle-based constraint. No explicit construction or use was found in the supplied sources.	None unless additional, unsupplied code consumes it.
<code>enabled</code>	Intended on/off switch for an angle constraint.	No visible effect in the supplied implementation.
<code>theta_star_deg</code>	Presumed target angle in degrees for a directional or junction-angle constraint.	No visible effect in the supplied implementation.
<code>delta</code>	Presumed tolerance or smoothing width for the angle constraint.	No visible effect in the supplied implementation.
<code>overlap</code>	Placeholder block for overlap-related constraints. No corresponding constructor or use was visible.	None unless unsupplied code implements it.
<code>enabled</code>	Intended on/off switch for an overlap constraint.	No visible effect in the supplied implementation.

4. Generic per-stage parameters

The following keys are valid inside each entry of `orientation_run`, `convergence_run`, and, by design analogy, the additive sub-stages `orientation_stage`, `convergence_relevant`, and `convergence_combined`. These keys are read in `run_optimization_stage()`.

Table 7: Technical stage-level parameters

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
<code>optimizer</code>	<code>orientation_run - optimizer</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> around lines 893–900	string; at least <code>ipopt</code> ; SciPy backend also supported	<code>ipopt</code>
<code>max_iter</code>	<code>orientation_run - max_iter</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 895	integer <code>> 0</code>	100
<code>tolerance</code>	<code>orientation_run - tolerance</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 893	positive float	<code>1e-6</code>
<code>use_constraints</code>	<code>orientation_run - use_constraints</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 890; constraints built via <code>build_constraints_from_config()</code>	boolean	<code>true</code>
<code>fixed_r</code>	<code>orientation_run - fixed_r</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 891; bounds assembled by <code>build_bounds()</code>	float or null	<code>None</code>
<code>reward_only</code>	<code>orientation_run - reward_only</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 896; also checked by <code>build_constraints_from_config()</code> around line 1235	boolean	<code>false</code>
<code>check_derivatives</code>	<code>orientation_run - check_derivatives</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 897	boolean	<code>false</code>
<code>use_first_derivative</code>	<code>orientation_run - use_first_derivative</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 898	boolean	<code>false</code>
<code>plot</code>	<code>orientation_run - plot</code> etc.	<code>optimize.py</code> , <code>optimization_definition.py</code> around lines 307–313; consumed in objective around lines 366–444	boolean or array containing objective and/or derivative	<code>true</code>
<code>make_gif</code>	<code>orientation_run - make_gif</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 892; GIF helper functions earlier in file	boolean	<code>false</code>

Parameter Name	JSON Path / Section	File + approx. location	Datatype / values	Default
<code>smaller_box</code>	<code>orientation_run - smaller_box</code> etc.	<code>optimize.py</code> , <code>run_optimization_stage()</code> line around 899; interacts with <code>build_bounds()</code>	boolean	<code>false</code>
<code>transition</code>	<code>orientation_run - transition</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	float	inherited from <code>global.transition</code> if absent
<code>extension</code>	<code>orientation_run - extension</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	float	inherited from <code>global</code>
<code>boundary</code>	<code>orientation_run - boundary</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	string	inherited from <code>global</code>
<code>combine</code>	<code>orientation_run - combine</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	string	inherited from <code>global</code>
<code>p</code>	<code>orientation_run - p</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	float / integer	inherited from <code>global</code>
<code>beta</code>	<code>orientation_run - beta</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	float	inherited from <code>global</code>
<code>N</code>	<code>orientation_run - N</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	integer	inherited from <code>global</code>
<code>k</code>	<code>orientation_run - k</code> etc.	<code>optimize.py</code> , stage override loop lines 868–870	float	inherited from <code>global</code>
<code>n</code>	<code>orientation_run - n</code> etc.	<code>optimize.py</code> , stage override loop lines 871–875	two-integer array	inherited from <code>global</code>
<code>bounds</code>	<code>orientation_run - bounds</code> etc.	<code>optimize.py</code> , stage override loop lines 876–877	object with <code>x</code> , <code>y</code> intervals	inherited from <code>global</code>

Table 8: Descriptions and effects of stage-level parameters

Parameter Name	Description / meaning	Typical effect
<code>optimizer</code>	Optimization backend selector. The project context and imports show support for IPOPT and SciPy-based optimization. Example uses <code>ipopt</code> .	Determines numerical algorithm, available derivative usage, convergence behavior, and practical robustness.
<code>max_iter</code>	Maximum number of iterations for the stage solver.	Higher values allow more progress but increase runtime; low values can be useful in coarse continuation stages.
<code>tolerance</code>	Convergence tolerance passed to the selected optimizer. Exact interpretation depends on backend.	Smaller tolerances aim for more accurate convergence but increase runtime and may expose conditioning problems.

Parameter Name	Description / meaning	Typical effect
<code>use_constraints</code>	Enables use of the configured nonlinear constraints for this stage.	Useful for early loose exploration without constraints, followed by constrained cleanup or final convergence.
<code>fixed_r</code>	If provided, the radius variable of each feature is fixed to this value through variable bounds.	Decouples pose/orientation updates from thickness optimization; often stabilizes early stages.
<code>reward_only</code>	Stage mode flag. In the supplied pipeline, orientation stages are typically run in this mode, meaning the objective formulation and/or constraint handling differs from full convergence stages.	Separates coarse directional alignment from full residual minimization; usually improves staged robustness.
<code>check_derivatives</code>	Diagnostic flag for verifying analytical derivatives against numerical checks.	Valuable during method development; expensive and usually disabled in production runs.
<code>use_first_derivative</code>	Flag that appears to disable or alter second-derivative usage, forcing a first-derivative-driven optimization path.	Can improve robustness if exact Hessians are unreliable or too expensive, at the cost of slower convergence.
<code>plot</code>	Controls diagnostic plot generation. <code>true</code> creates objective plots, <code>false</code> disables plotting, and a list may contain <code>objective</code> , <code>derivative</code> , or both.	Provides visual feedback on optimization progress and sensitivities; additional plots increase runtime and storage requirements.
<code>make_gif</code>	If enabled, the stage writes frames and assembles them into an animation.	Useful for visual debugging and reporting; increases I/O overhead.
<code>smaller_box</code>	Stage-local flag that tightens variable bounds to a reduced domain box.	Can regularize early search and prevent wandering endpoints, but may be too restrictive if the target structure spans the full domain.
<code>transition</code>	Stage-local override of <code>global.transition</code> .	Supports continuation from diffuse to sharp geometry across stages.
<code>extension</code>	Stage-local override of <code>global.extension</code> .	Enables coarse-to-fine continuation in effective feature influence or box extension.
<code>boundary</code>	Stage-local override of the boundary transition law.	Useful when starting with a very smooth law and switching to a sharper law later.
<code>combine</code>	Stage-local override of the aggregation operator.	Allows continuation in aggregation rule, e.g. from smoother blending toward more max-like composition.
<code>p</code>	Stage-local override of p-norm exponent.	Common continuation mechanism: gradually increase <code>p</code> to sharpen overlap handling.
<code>beta</code>	Stage-local override of softmax / soft-cap sharpness.	Common continuation mechanism: gradually increase <code>beta</code> to approximate harder selection or capping.
<code>N</code>	Stage-local override of the smoothstep or cap order.	Permits controlled changes in transition smoothness between stages.
<code>k</code>	Stage-local override of the sigmoid sharpness parameter.	Allows stage-specific tuning of boundary steepness.
<code>n</code>	Stage-local grid resolution override. The stage explicitly resets <code>sp.glob_n</code> , <code>dx</code> , and <code>dy</code> .	Enables multiresolution continuation: coarse stages run cheaply; fine stages improve final fidelity.

Parameter Name	Description / meaning	Typical effect
bounds	Stage-local domain bounds override.	Supports box continuation or local stage-specific embedding of the target field.

5. Standard stage containers

Table 9: Technical details of the standard stage containers

Parameter Name	JSON Path / Section	File + approximate location	Datatype / values	Default
orientation_run	orientation_run	optimize.py, loop starting around line 2205	array of stage objects	empty list
convergence_run	convergence_run	optimize.py, loop starting around line 2223	array of stage objects	empty list

Table 10: Description and effects of the standard stage containers

Parameter Name	Description / meaning	Typical effect
orientation_run	Sequence of early stages intended to align feature positions/orientations before the final residual-fitting stages. The pipeline typically augments each entry with inherited <code>global</code> settings and the shared <code>constraints</code> block.	Helps avoid poor local minima by first solving an easier or more regularized subproblem.
convergence_run	Sequence of later stages intended to reduce the full reconstruction error after orientation has stabilized.	Main quality-improvement schedule; often uses smaller transition width, lower extension, and tighter tolerance than orientation stages.

6. Heuristics block (heuristics)

Table 11: Technical details of the heuristics block

Parameter Name	JSON Path / Section	File + approximate location	Datatype / values	Default
enable_merge	heuristics - enable_merge	optimize.py, apply_heuristics() and heuristic pipeline	boolean	false
ARmin	heuristics - ARmin	optimize.py, passed into identify_removal_candidates(...,ARlim,URLim)	float ≥ 0	0.005
URmin	heuristics - URmin	optimize.py, passed into identify_removal_candidates(...,ARlim,URLim)	float ≥ 0	0.012
theta_lim_deg	heuristics - theta_lim_deg	optimize.py, compute_angle_adjacent_groups() and collinearity utilities	float (degrees)	10.0
min_dist	heuristics - min_dist	optimize.py, grouping logic in compute_angle_adjacent_groups()	float ≥ 0	0.15
use_segment_distance	heuristics - use_segment_distance	optimize.py, adjacency computation routines	boolean	false
simple	heuristics - simple	optimize.py, apply_heuristics() and simple removal pipeline	object	disabled / absent
enabled	heuristics - simple.enabled	optimize.py, simple heuristic branch (optional configuration)	boolean	false
min_r	heuristics - simple.min_r	optimize.py, identify_simple_removals()	float ≥ 0	0.0001
min_seglen	heuristics - simple.min_seglen	optimize.py, identify_simple_removals()	float ≥ 0	0.001

Table 12: Description and effects of the heuristics block

Parameter Name	Description / meaning	Typical effect
enable_merge	Enables post-optimization merging of nearby and similarly oriented features after pruning steps.	Reduces redundancy and simplifies geometry, but may remove useful degrees of freedom if applied too aggressively.
ARmin	Minimum normalized area ratio required to retain a feature during pruning.	Removes very small or weak features with negligible contribution to the overall shape.

Parameter Name	Description / meaning	Typical effect
URmin	Minimum uniqueness ratio defining how much of a feature is not already covered by others.	Suppresses redundant overlapping features; typically improves compactness before refinement.
theta_lim_deg	Angular threshold for grouping or merging features based on directional similarity.	Controls aggressiveness of directional merging; lower values enforce stricter separation.
min_dist	Spatial threshold for adjacency in merge/group detection, either endpoint-based or segment-based depending on configuration.	Determines how close features must be before being treated as part of the same structure.
use_segment_distance	Switch between endpoint distance and full segment-to-segment distance for adjacency decisions.	Segment-based mode improves geometric robustness for elongated features but is more expensive.
simple	Optional early-stage cleanup block applying hard thresholds before AR/UR-based heuristics.	Removes obviously degenerate features before more advanced pruning and merging logic.
min_r	Minimum radius threshold used in the simple heuristic removal pass.	Eliminates extremely thin or numerically unstable features.
min_seglen	Minimum segment length threshold used in the simple heuristic removal pass.	Removes nearly collapsed or point-like features prior to further optimization.

7. Additive refinement block (additive)

Table 13: Technical details of the additive refinement block

Parameter Name	JSON Path / Section	File + approximate location	Datatype / values	Default
enable	additive - enable	optimize.py, greedy_additive_refinement() and pipeline call site	boolean	false
max_additions	additive - max_additions	optimize.py, greedy_additive_refinement()	integer ≥ 0	3
residual_threshold	additive - residual_threshold	optimize.py, _make_residual(S_{tgt}, S_{cur}, thr) in additive loop	float ≥ 0	0.5
seed_radius	additive - seed_radius	optimize.py, additive initialization logic	float > 0	0.05
fixed_r	additive - fixed_r	optimize.py, additive stage setup and build_bounds() interaction	float or null	0.05
l_max	additive - l_max	optimize.py, additive seeding constraints	float > 0	0.3
improve_min_rel	additive - improve_min_rel	optimize.py, additive stopping logic	float ≥ 0	0.001
improve_min_abs	additive - improve_min_abs	optimize.py, additive stopping logic	float ≥ 0	0
orientation_stage	additive - orientation_stage	optimize.py, greedy_additive_refinement()	object	absent if disabled
convergence_relevant	additive - convergence_relevant	optimize.py, greedy_additive_refinement()	object	absent if disabled
convergence_combined	additive - convergence_combined	optimize.py, greedy_additive_refinement()	object	absent if disabled

Table 14: Description and effects of the additive refinement block

Parameter Name	Description / meaning	Typical effect
enable	Master switch for greedy additive refinement. Controls whether new features can be inserted after the main optimization pipeline.	Enables model growth only when necessary; otherwise keeps model size fixed.
max_additions	Upper bound on the number of additive refinement cycles or inserted features.	Limits runtime and final model complexity.

Parameter Name	Description / meaning	Typical effect
<code>residual_threshold</code>	Threshold on the residual field used to determine where new features should be seeded.	Controls aggressiveness of refinement; higher values focus on strong errors, lower values increase coverage.
<code>seed_radius</code>	Initial radius assigned to newly created features before local optimization.	Determines initial influence area of new features; affects stability of insertion.
<code>fixed_r</code>	Optional fixed radius used during early additive sub-stages.	Stabilizes initial placement and orientation of newly added features.
<code>l_max</code>	Maximum allowed segment length for newly inserted features during additive refinement.	Prevents unrealistic or overly extended new features.
<code>improve_min_rel</code>	Minimum relative improvement required to continue or accept additive refinement steps.	Stops refinement when gains become negligible in relative terms.
<code>improve_min_abs</code>	Minimum absolute improvement required to continue or accept additive refinement steps.	Prevents excessive refinement when absolute objective improvement is small.
<code>orientation_stage</code>	Local stage configuration applied immediately after inserting a new feature, focusing on orientation and placement.	Improves initial alignment of newly added features before full optimization.
<code>convergence_relevant</code>	Stage that optimizes only the newly added feature or locally relevant subset.	Reduces local error introduced by insertion before global re-optimization.
<code>convergence_combined</code>	Final stage after insertion that re-optimizes the full system.	Restores global consistency and adapts all features to the updated configuration.

8. Nested additive stage parameters

Table 15: Technical details of nested additive stage parameters

Parameter Name	JSON Path / Section	File + approximate location	Datatype / values	Default
optimizer	additive - orientation_stage - optimizer, additive - convergence_relevant - optimizer, additive - convergence_combined - optimizer	optimize.py, additive loop calling run_optimization_stage()	string	ipopt
max_iter	additive.*.max_iter	optimize.py, additive stage execution	integer	50
tolerance	additive.*.tolerance	optimize.py, additive stage execution	float	0.000001
transition	additive - convergence_combined - transition (and other additive stages if defined)	optimize.py, stage override handling	float	0.05
combine	additive.*.combine	optimize.py, additive stage override propagation	string	"p-norm"
p	additive.*.p	optimize.py, additive stage override propagation	float / integer	9
beta	additive.*.beta	optimize.py, additive stage override propagation	float	12
extension	additive.*.extension	optimize.py, additive stage override propagation	float	0
make_gif	additive.*.make_gif	optimize.py, additive stage logging and visualization hooks	boolean	false

Table 16: Description and effects of nested additive stage parameters

Parameter Name	Description / meaning	Typical effect
optimizer	Optimization backend used for each additive sub-stage (orientation, local correction, full recombination).	Determines solver behavior, convergence properties, and derivative handling during refinement.

Parameter Name	Description / meaning	Typical effect
<code>max_iter</code>	Maximum iteration budget for the respective additive sub-stage.	Controls computational effort per inserted feature or refinement cycle.
<code>tolerance</code>	Convergence criterion for stopping the additive sub-stage optimizer.	Smaller values increase accuracy but may increase runtime and sensitivity to conditioning.
<code>transition</code>	Local override of the boundary transition width for additive sub-stages.	Typically reduced in later stages to sharpen geometry during final refinement.
<code>combine</code>	Aggregation rule override for feature combination during additive refinement.	Allows smoother early blending and sharper late-stage composition within additive updates.
<code>p</code>	p-norm exponent override controlling max-like behavior in aggregation.	Higher values increase competition between features and sharpen structural separation.
<code>beta</code>	Sharpness parameter for softmax or soft-cap aggregation used in additive stages.	Controls transition from smooth to near-hard selection behavior.
<code>extension</code>	Overrides geometric extension or effective influence range per additive stage.	Enables broader exploration during insertion and tighter fitting during finalization.
<code>make_gif</code>	Enables visualization output for the corresponding additive sub-stage.	Produces diagnostic animations at the cost of increased I/O overhead.

9. Inheritance and override behavior

- The `global` block defines the baseline numerical environment. Before each stage, the framework copies stage-local keys from the set `transition`, `extension`, `extension_kind`, `boundary`, `N`, `k`, `combine`, `p`, `n`, `bounds`, `beta` into `pill.glob`. Therefore, these fields behave as *stage-local overrides* of the global defaults.
- The `constraints` block is shared across stages. Whether it is active in a particular stage is controlled by the stage flag `use_constraints`.
- Orientation stages are typically configured as easier or more regularized stages (often with `fixed_r`, larger `extension`, and looser tolerances), while convergence stages tighten these settings.
- The additive sub-stages follow the same stage semantics as the standard pipeline. In practice, they should be interpreted as local reuses of the same stage API inside the greedy refinement loop.